Week 8

1. Single Inheritance

Problem Statement:

Write a Java program to demonstrate **single inheritance** by creating a `Person` superclass containing the name and age of a person. Create a `Student` subclass containing roll number and branch. Inherit the properties of `Person` and display the complete student details.

```
class Person{
        String name;
        int age;
        void details(){
                System.out.println("Name is "+name);
                System.out.println("Age is "+age);
        }
}
class Student extends Person{
        int roll;
        String branch;
        void cred(){
                System.out.println("Roll Number: "+roll);
                System.out.println("Branch : "+branch);
        }
}
class Single{
        public static void main(String[] args){
                Student newStudent = new Student();
                newStudent.name = "Prazod";
                newStudent.age = 18;
                newStudent.roll = 98;
                newStudent.branch = "AIML";
                newStudent.details();
                newStudent.cred();
        }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac Single.java

C:\Users\ITLAB2\Desktop\25EU02098>java Single
Name is Prazod
Age is 18
Roll Number: 98
Branch : AIML
```

2. Using super Keyword

Problem Statement:

Write a Java program to demonstrate the use of the super **keyword**. Create a Vehicle superclass with a speed variable and a display() method. Create a Car subclass with a model variable and override the display() method. Use super to access the superclass variable and method.

```
class Vehicle {
    int speed = 100;

    void display() {
        System.out.println("Inside Vehicle class");
    }
}

class Car extends Vehicle {
    String model = "Sedan";

    void display() {
        super.display(); // 1. Calls superclass method
        System.out.println("Model: " + model);
        System.out.println("Speed: " + super.speed); // 2. Accesses superclass variable
    }
}

public class Main2 {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main2.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main2
Inside Vehicle class
Model: Sedan
Speed: 100
```

3. Multilevel Inheritance

Problem Statement:
Write a Java program to demonstrate **multilevel inheritance** by creating three classes: Person, Employee, and Manager. The Person class should contain personal details, Employee should contain employee details, and Manager should contain department details. Display all the details using inherited properties.

```
class Person {
    String name = "John";
}

class Employee extends Person {
    int empId = 101;
}

class Manager extends Employee {
    String department = "IT";

    void display() {
        // Accessing inherited properties directly
        System.out.println("Name: " + name);
        System.out.println("ID: " + empId);
        System.out.println("Dept: " + department);
    }
}

public class Main3 {
    public static void main(String[] args) {
        Manager mgr = new Manager();
        mgr.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main3.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main3
Name: John
ID: 101
Dept: IT
```

4. Method Overriding

Problem Statement:
Write a Java program to demonstrate **method overriding** by creating an Animal superclass with a sound() method. Create subclasses Dog and Cat that override the sound() method with their respective sounds. Display the appropriate sound for each animal.

```
class Animal {
    void sound() {
        System.out.println("Generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark!");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow!");
    }
}

public class Main4 {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        Cat myCat = new Cat();

        myDog.sound(); // Calls Dog's overridden method
        myCat.sound(); // Calls Cat's overridden method
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main4.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main4
Bark!
Meow!
```

5. Dynamic Method Dispatch

Problem Statement:

Write a Java program to demonstrate **dynamic method dispatch** using a superclass Shape and subclasses Circle, Rectangle, and Triangle. Override the draw() method in each subclass and use a superclass reference to invoke the appropriate overridden method at runtime.

```
class Shape {
    void draw() {
        System.out.println("Drawing a generic shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}

public class Main5 {
    public static void main(String[] args) {
        // Creating a superclass reference
        Shape myShape;

        // Dynamic Method Dispatch: The method called is determined
        // by the actual object type at runtime, not the reference type.

        myShape = new Circle();
        myShape.draw();

        myShape = new Rectangle();
        myShape.draw();

        myShape = new Triangle();
        myShape.draw();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main5.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main5
Drawing a Circle
Drawing a Rectangle
Drawing a Triangle
```

6. super with Method Overriding

Problem Statement:

Write a Java program to demonstrate the use of super **with method overriding**. Create an Employee class with a display() method and derive a Manager class that overrides the method. Use super.display() in the subclass to invoke the superclass implementation before displaying the manager details.

```
class Employee {
    String name = "Alex";

    void display() {
        System.out.println("Employee Name: " + name);
    }
}

class Manager extends Employee {
    String department = "IT";

    void display() {
        super.display(); // Invokes the superclass (Employee) method first
        System.out.println("Manager Department: " + department);
    }
}

public class Main6 {
    public static void main(String[] args) {
        Manager mgr = new Manager();
        mgr.display();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main6.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main6
Employee Name: Alex
Manager Department: IT
```

7. Multilevel Inheritance for Salary Calculation

Problem Statement:

Write a Java program to create a multilevel inheritance hierarchy Employee → Developer → SeniorDeveloper. Store basic salary in the Employee class, programming allowance

in the Developer class, and project allowance in the SeniorDeveloper class. Calculate and

display the total salary.

```
class Employee {
    double basicSalary = 50000.0;
}

class Developer extends Employee {
    double programmingAllowance = 15000.0;
}

class SeniorDeveloper extends Developer {
    double projectAllowance = 20000.0;

    void displayTotalSalary() {
        // Accessing inherited properties from both parent classes
        double totalSalary = basicSalary + programmingAllowance + projectAllowance;

        System.out.println("Basic Salary: ₹" + basicSalary);
        System.out.println("Programming Allowance: ₹" + programmingAllowance);
        System.out.println("Project Allowance: ₹" + projectAllowance);
        System.out.println("-----------------------------");
        System.out.println("Total Salary: ₹" + totalSalary);
    }
}

public class Main7 {
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.displayTotalSalary();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main7.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main7
Basic Salary: ?50000.0
Programming Allowance: ?15000.0
Project Allowance: ?20000.0
-----------------------------
Total Salary: ?85000.0
```

8. Dynamic Method Dispatch for Bank Accounts

Problem Statement:

Write a Java program to demonstrate **dynamic method dispatch** using a superclass BankAccount with a calculateInterest() method. Create SavingsAccount and CurrentAccount subclasses with different implementations of the method. Use a superclass reference to calculate interest for different account types.

```
class BankAccount {
    void calculateInterest() {
        System.out.println("Calculating standard interest rate.");
    }
}

class SavingsAccount extends BankAccount {
    void calculateInterest() {
        System.out.println("Savings Account Interest Rate: 4.5%");
    }
}

class CurrentAccount extends BankAccount {
    void calculateInterest() {
        System.out.println("Current Account Interest Rate: 0.0%");
    }
}

public class Main8 {
    public static void main(String[] args) {
        // Creating a superclass reference
        BankAccount account;

        // Dynamic Method Dispatch: Calling SavingsAccount method
        account = new SavingsAccount();
        account.calculateInterest();

        // Dynamic Method Dispatch: Calling CurrentAccount method
        account = new CurrentAccount();
        account.calculateInterest();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main8.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main8
Savings Account Interest Rate: 4.5%
Current Account Interest Rate: 0.0%
```

9. Constructor Execution in Multilevel Inheritance

Problem Statement:
Write a Java program to demonstrate the **order of constructor execution in multilevel inheritance** using the classes Person, Student, and GraduateStudent. Use constructors in all three classes and display messages to show the order in which the constructors are executed.

```
class Person {
    Person() {
        System.out.println("Person constructor executed.");
    }
}

class Student extends Person {
    Student() {
        System.out.println("Student constructor executed.");
    }
}

class GraduateStudent extends Student {
    GraduateStudent() {
        System.out.println("GraduateStudent constructor executed.");
    }
}

public class Main9 {
    public static void main(String[] args) {
        System.out.println("Creating GraduateStudent object:");

        // Instantiating the lowest subclass triggers the constructor chain
        // from the top-most superclass down to the subclass.
        GraduateStudent gs = new GraduateStudent();
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main9.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main9
Creating GraduateStudent object:
Person constructor executed.
Student constructor executed.
GraduateStudent constructor executed.
```

10. Banking Application Using Inheritance

Problem Statement:

Write a Java program to develop a simple **banking application using inheritance**. Create a BankAccount superclass with account number and balance. Derive SavingsAccount and CurrentAccount classes and provide suitable methods for deposit, withdrawal, and interest calculation. Demonstrate inheritance and method overriding.

```
class BankAccount {
    int balance = 1000;

    void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + ", New Balance: " + balance);
    }

    void withdraw(int amount) {
        balance -= amount;
        System.out.println("Withdrawn: " + amount + ", New Balance: " + balance);
    }

    void calculateInterest() {
        System.out.println("Calculating standard interest.");
    }
}

class SavingsAccount extends BankAccount {
    @Override
    void calculateInterest() {
        System.out.println("Savings Account gets 5% interest.");
    }
}

class CurrentAccount extends BankAccount {
    @Override
    void calculateInterest() {
        System.out.println("Current Account gets 0% interest.");
    }
}

public class Main10 {
    public static void main(String[] args) {
        System.out.println("--- Savings Account ---");
        SavingsAccount savings = new SavingsAccount();
        savings.deposit(500);   // Inherited method
        savings.withdraw(200);  // Inherited method
        savings.calculateInterest(); // Overridden method

        System.out.println("\n--- Current Account ---");
        CurrentAccount current = new CurrentAccount();
        current.deposit(1000);
        current.calculateInterest(); // Overridden method
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Main10.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Main10
--- Savings Account ---
Deposited: 500, New Balance: 1500
Withdrawn: 200, New Balance: 1300
Savings Account gets 5% interest.

--- Current Account ---
Deposited: 1000, New Balance: 2000
Current Account gets 0% interest.
```